Glossary

Level 2 words from A to Z, in plain language

The week in brackets tells you where the word is taught, so you can look back in your workbook. Words in `this font` are things you type in Python.

A

and Joins two conditions. Both must be True: `score > 5 and lives > 0`. (week 1)

append Adds an item to the end of a list: `names.append("Zoe")`. (week 3)

argument The value you put in the brackets when you call a function. In `area(3, 4)` the arguments are 3 and 4. (week 2)

`assert` Checks that something is True. If it is not, the test fails and shows your message. (week 8)

attribution Saying who made the art, sound or code you used. Giving credit. (week 9)

B

bool The type for True or False. (week 1)

`break` Leaves a loop straight away. (week 4)

bug A mistake in a program. Every programmer makes them; finding them is part of the fun.

C

Caesar cipher A secret code that shifts every letter along the alphabet by the same amount. (week 5)

code review Reading someone's code to ask questions, find bugs and say what is good. (week 10)

collision When two things in a game touch. We check it with `distance`. (week 9)

comment A note for people, after a #. Python ignores it. (week 2)

compound condition Two or more conditions joined with `and`, `or` or `not`. (week 1)

D

data type The kind of value: int, float, str or bool. Also just called a type. (week 1)

debugging Finding and fixing bugs with a strategy: read the traceback, print-debugging, rubber duck. (week 6)

decomposition Breaking a big problem into small parts you can solve one at a time. (week 10)

dictionary (dict) A collection that stores values under names (keys): `{"lives": 3}`. (week 5)

docstring A comment in triple quotes as the first line of a function, saying what it does. (week 2)

driver In pair programming, the person at the keyboard.

E

edge case A test at the very edge of what is allowed: zero, an empty list, a wall. (week 8)

encryption Scrambling a message so only people with the key can read it. (week 5)

event Something that happens, like a key press, a click or a timer going off. (week 9)

event handler The function that runs when an event happens. (week 9)

F

f-string A string with an `f` in front that fills in values: `f"Score: {score}"`. (week 1)

file Data saved on the computer, so it is still there after the program stops. (week 7)

`FileNotFoundError` The error you get when you try to read a file that does not exist (yet). (week 7)

float The type for decimal numbers like 3.5. (week 1)

flowchart A drawing of a program's steps with boxes and arrows. (week 10)

function A named block of code you can use again. Made with `def`. (week 2)

G

game loop The part of a game that runs again and again: move, check, draw. (week 4, 9)

game state All the values that describe the game right now: score, lives, positions. (week 9)

I

index The position of an item in a list. The first item has index `0`. (week 3)

`IndexError` The error you get when you ask for an index that is not in the list. (week 3)

infinite loop A loop that never stops. Press the red Stop button. (week 4)

input validation Checking that what the user typed makes sense, and asking again if not. (week 4)

int The type for whole numbers like 7 or -3. (week 1)

item One value inside a list. Also called an element. (week 3)

K

key The name you use to look up a value in a dictionary. (week 5)

`KeyError` The error you get when a key is not in the dictionary. Check the spelling! (week 5)

L

`len` Tells you how many items are in a list (or letters in a string). (week 3)

list A collection of values in order: `["red", "blue"]`. (week 3)

`listen` Makes the turtle window pay attention to the keyboard. (week 9)

M

mode How a file is opened: `"r"` read, `"w"` write (starts empty), `"a"` append (adds to the end). (week 7)

N

navigator In pair programming, the person who reads the task, thinks ahead and spots bugs.

nested loop A loop inside another loop. Great for grids and pixel art. (week 4)

normal case A test with ordinary, everyday values. (week 8)

`not` Turns True into False and False into True. (week 1)

O

`onkey` Tells turtle which function to run when a key is pressed. (week 9)

`ontimer` Tells turtle to run a function after some milliseconds. (week 9)

open license Permission from a creator that lets others use their work, often if they give credit. (week 9)

`or` Joins two conditions. At least one must be True. (week 1)

P

parameter A name in a function's brackets that receives a value: `def area(width, height)`. (week 2)

personal information Anything that tells who or where you are: real name, address, school, birthday, photos. (week 1)

playtest Letting someone try your game while you watch and take notes. (week 11)

print-debugging Adding `print` lines to see what your variables hold. (week 6)

pseudocode Planning a program in plain words before writing Python. (week 10)

pytest A tool that finds your `test_` functions and runs them all. (week 8)

R

`random.choice` Picks one random item from a list. (week 3)

`return` Hands a value back from a function to the code that called it. (week 2)

return value The value a function hands back. (week 2)

rubber duck debugging Explaining your code line by line out loud (to a toy or a friend) until you spot the bug. (week 6)

S

scope Where a variable can be used. Variables made inside a function stay inside it. (week 2)

`split` Cuts a string into a list of words: `"a b".split()` gives `["a", "b"]`. (week 7)

str (string) The type for text, written in quotes: `"hello"`. (week 1)

`strip` Removes spaces and new-line characters from both ends of a string. (week 7)

T

test A small function that checks your code works, using `assert`. (week 8)

trace table A table on paper where you write how variables change, step by step. (week 6)

traceback The error report Python shows. Read the last line first. (week 6)

`TypeError` The error you get when you mix types that do not fit, like `"5" + 5`. (week 1)

V

value A piece of data, like 7, `"hi"` or True. In a dictionary, what is stored under a key. (week 1, 5)

variable A name that stores a value. Choose clear names like `lives`, not `x`. (week 1)

W

`while` loop Repeats while a condition is True. Something inside must change so it can stop. (week 4)

`with` Opens a file and closes it again safely when the block ends. (week 7)

More words from your workbook

.items() Gives every key together with its value, so a loop can use both. (week 5)

accessibility designing so that everyone can use it, including people who see colors differently or read more slowly. (week 11)

accessible Designed so that everyone can use it, including people who see colors differently. (week 3)

bug list a list of problems found in testing, sorted so the most important ones get fixed first. (week 11)

bug report A short note: what went wrong, where, and what you changed. (week 6)

core loop the part of a game or app that repeats over and over; the smallest version that works. (week 10)

data privacy Deciding what information about people is saved, who can see it, and keeping it safe. (week 7)

design decision a choice you made on purpose, and the reason for it. (week 12)

encode / decode Turn a message into secret code / turn it back again. (week 5)

expected vs actual What the test says should happen, and what really happened. (week 8)

frame One tiny step of a game: everything moves

a little, then it is drawn. (week 9)

gallery walk visiting projects one by one, like in a museum, and leaving feedback. (week 12)

license / attribution The rules for using someone's work / giving them credit for it. (week 9)

milestone a checkpoint on the way to the finished project, like M2 "core loop works". (week 10)

None Python's word for "nothing". A function with no `return` gives back `None`. (week 2)

off-by-one A bug where a number is one too big or one too small, like using `len` as the last index. (week 3)

open / with `open` gets a file ready to use; `with` closes it again automatically. (week 7)

pitch a short talk that explains what your project is and why it is interesting. (week 12)

polish small finishing touches that make a working program clearer and nicer to use. (week 11)

read / write / append The modes `"r"`, `"w"` and `"a"`: look, replace everything, or add to the end. (week 7)

showcase an event where makers show their work and explain it. (week 12)

version one stage of a project; version 2 is the next, improved one. (week 12)